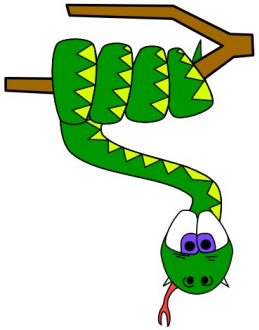
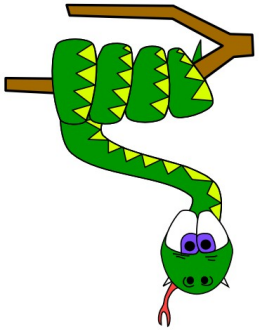


Basics of Python



Identifers

indentifiers

- Identifers are names used for identifying functions, classes, modules or variables.
- They can start with an underscore or with letter (uppercase or lowercase).
- Can be followed by numbers, letters or underscores, but can’t be followed by special characters (**@, $, %**).
- You can’t use reserved words (listed later) as an identifer.

indentifiers

- identifiers that start with underscore usually have special meaning:

- Single leading underscore (_name) - identifier is meant to be private.
  It is just a convention to signal that a variable should not be accessed directly outside the class.

- Single Trailing Underscore (var_) - There is only one reason to use single trailing underscore which is to avoid conflicts with Python keywords.

- Two leading underscores (__name) - identifier is private and you can't call it from outside of the class

- Two leading and two ending underscores (__name__) - identifier is reserved for special use in the language.
  Example : _ _**init**_ _ is for object constructors

indentifiers

```
class Test:
        var1=10
       _ _var2=20

ob=Test()
print(ob.var1)
print(ob._ _var2)
```

OUTPUT

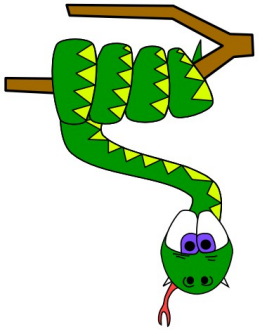
```
10

Traceback (most recent call last):
  File "myprg.py", line 7, in <module>
    print(ob.__var2)
AttributeError: 'Test' object has no attribute '__var2'
```

indentifiers

- Python is a case-sensitive language. X and x are two different identifiers.

- **Examples of correct identifiers:** identifier, _identifier, Identifier, IdenTifier, _Identifier, identifier__, Identifier5430

- **Examples of incorrect identifiers:** any reserved word, identifier@, $identifier, identif%er…



variables

CHINTECH

SMP

variables

- Variables are containers for storing data values.

- Unlike other programming, variables do not need to be declared with any particular data type, and can even change the **type** after they have been set.

Example:

```
x = 4 # x is of type int

x = "Sally" # x is now of type str

print(x)
```

String variables can be declared either by using single or double quotes:

```
x = "John"
```

variables

Variable Names

A variable can have a short name (like x and y) or a more descriptive name (age, carname, total_volume).

Rules for Python variables:

- A variable name must start with a letter or the underscore character
- A variable name cannot start with a number
- A variable name can only contain alpha-numeric characters and underscores (A-z, 0-9, and _)
- Variable names are case-sensitive (age, Age and AGE are three different variables)

variables

Assign Value to Multiple Variables

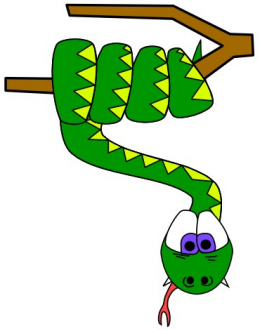
Python allows you to assign values to multiple variables in one line:

```
x, y, z = "Orange", "Banana", "Cherry"

print(x)

print(y)

print(z)
```

variables

And you can assign the same value to multiple variables in one line

```
x = y = z = "Orange"

print(x)

print(y)

print(z)
```

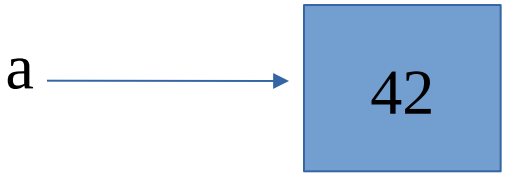


Python Data Types

Python Data Types

- In Python, every piece of data stored in a program is an object. Each object has an identity, a type (which is also known as its class), and a value

- a = 42 , an integer object is created with the value of 42, a is a name that refers to this specific location

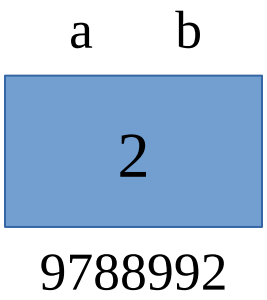


Python Data Types

- **id()** is a built-in function, returns object's location in memory
- **is** operator compares the identity of two objects
- **type()** returns the type of an object

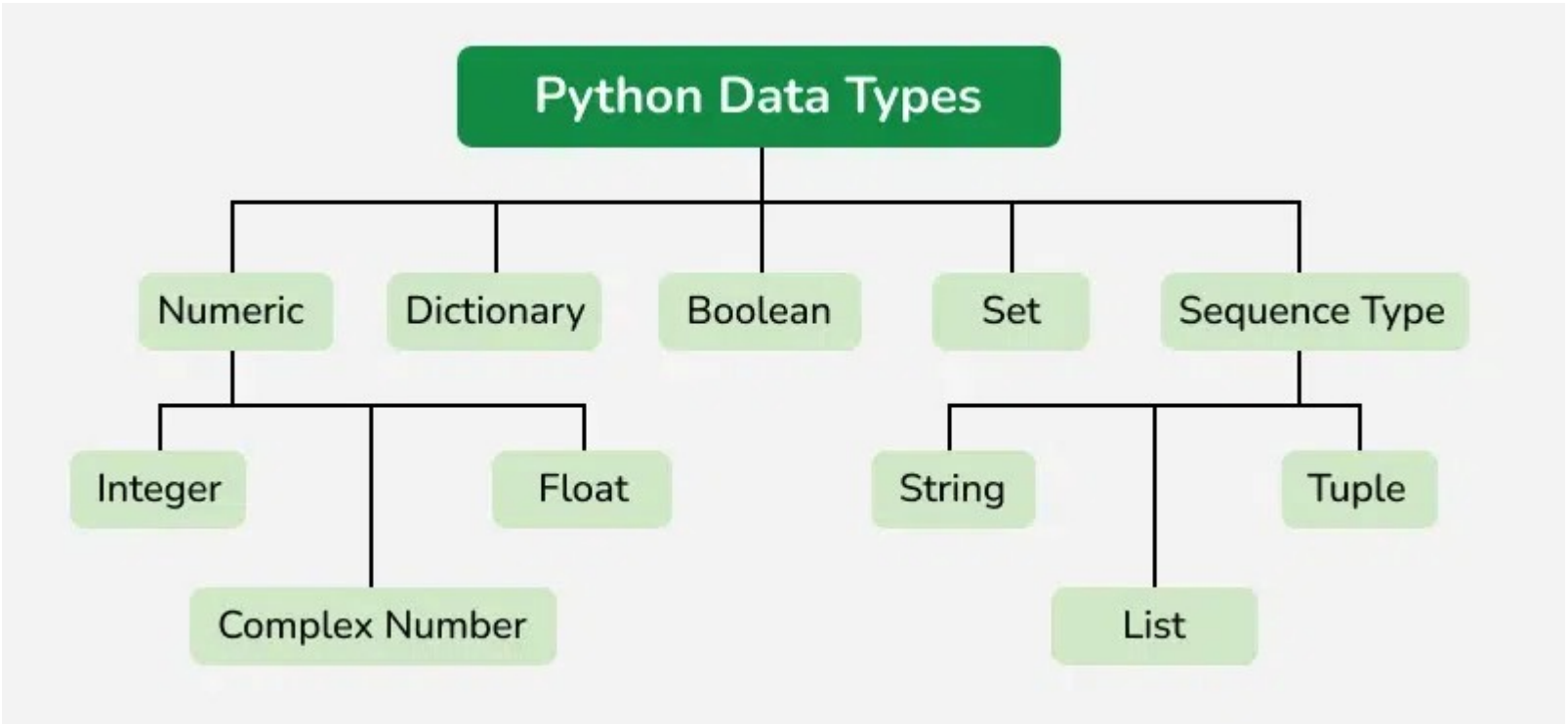
```
>>> a=2
>>> b=2
>>> id(a)
9788992
>>> id(b)
9788992
```

```
>>> a is b
True
>>>
>>> type(a)
<class 'int'>
```



Python Data Types

- Data types in Python define the type of value stored in a variable and determine the operations that can be performed on that data.
- Since Python treats everything as an object, each value is associated with a specific data type.
  - It helps the interpreter to understand how to store and process different kinds of data efficiently.
  - Enables correct operations by ensuring only valid actions are performed on compatible data types.


Python Data Types
Numeric
Dictionary
Boolean
Set
Sequence Type
Integer
Float
Complex Number
String
Tuple
List

Python Data Types

- Below code assigns different types of values to variable x.
- With each new assignment, the previous value is replaced and x changes to the new data type.

```
x = 50
x = 60.5
x = "Hello World"
x = ["chintech", "chala", "kannur"]
x = ("chintech", "chala", "kannur")
```

Python Data Types

Table 3.1 **Built-In Types for Data Representation**

Type Category	Type Name	Description
None	`type(None)`	The null object `None`
Numbers	`int`	Integer
	`long`	Arbitrary-precision integer (Python 2 only)
	`float`	Floating point
	`complex`	Complex number
	`bool`	Boolean (`True` or `False`)
Sequences	`str`	Character string
	`unicode`	Unicode character string (Python 2 only)
	`list`	List
	`tuple`	Tuple
	`xrange`	A range of integers created by `xrange()` (In Python 3, it is called `range`.)
Mapping	`dict`	Dictionary
Sets	`set`	Mutable set
	`frozenset`	Immutable set

Numeric Data Types

- Numeric data types are used to store numeric values. It can be an integer, floating number or even a complex number.

- Python supports three main numeric types: **Integers**, **Float**, **Complex**

```
a = 5
b = 5.0
c = 2 + 4j

print(type(a))
print(type(b))
print(type(c))
```

Output

```
<class 'int'>
<class 'float'>
<class 'complex'>
```

Numeric Data Types

Boolean Data Type

- Boolean data type represents one of two values: **True** or **False**. It is mainly used in conditions and comparisons and is represented by the bool class.

```
a = True
b = False

print(type(a))
print(type(b))
```

Output

```
<class 'bool'>
<class 'bool'>
```

Sequence Data Types

- A sequence is an ordered collection of items, which can be of similar or different data types.
- Elements in a sequence can be accessed using indexing.

Sequence Data Types

1. String

Strings are used to store text data.

A string is represented using the **str** class and can be created using single, double or triple quotes.

```
s = 'Welcome to Python Programming'
print(s)
print(type(s))

# access string with index
print(s[1])
print(s[5])
```

output

```
Welcome to Python Programming
<class 'str'>
e
m
```

Sequence Data Types

2. List

Lists are ordered and **mutable** collections used to store multiple items in a single variable.

Elements in a list can be of different data types and are accessed using indexing.

```
a = [1, 2, 3]
print(a)

b = ["Alex", 4, 165.2]
print(b[0])
print(b[2])
```

output

```
[1, 2, 3]
Alex
165.2
```

Sequence Data Types

3. Tuple

Tuples are ordered and immutable collections used to store multiple items in a single variable.

Once created, tuple elements cannot be modified and are accessed using indexing.

A tuple with a single element must include a trailing comma, otherwise Python treats it as a normal value instead of a tuple.

```
t1 = ('Alex', 20, 165.2)
print(t1[0])
print(t1[2])
```

output

```
Alex
165.2
```

Set Data Type

Sets are unordered and mutable collections used to store unique elements.

Since sets are unordered, elements cannot be accessed using indexing.

Elements are usually accessed by iterating through the set using a loop.

```
s1 = {"a", "a", "b", "c", "b"}
print(s1)

s2 = {"chintech", "chala", "chintech",1}
for i in s2:
    print(i)
```

output

```
{'b', 'a', 'c'}
Chintech
chala
1
```

Dictionary Data Type

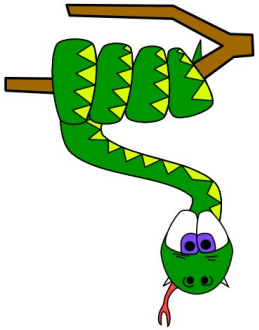
Dictionaries are used to store data in **key:value** pairs.

Each key in a dictionary must be unique and values are accessed using their keys with square brackets [] or get() method.

```
d = {1: 'Alex', 2: 'Alen', 3: 'Joe'}
print(d[1])
print(d.get(2))
```

output

```
Alex
Alen
```



keywords

keywords

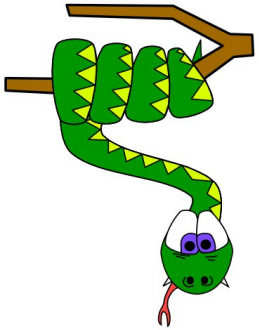
- Keywords are the reserved words in Python
- Python has a set of keywords that are reserved words that cannot be used as variable names, function names, or any other identifiers:
- Here is the list of reserved words you can’t use as identifiers:

Keywords in Python				
False	class	finally	is	return
None	continue	for	lambda	try
True	def	from	nonlocal	while
and	del	global	not	with
as	elif	if	or	yield
assert	else	import	pass	
break	except	in	raise	

keywords

```
def myfunction()
        print("hello how are you")

myfunction()
```

```
x=14
if x>10 and x<20:
  print (" x is between 10 an 20")
```



Statement & expressions

CHINTECH

SMP

statements and expressions

- **Statement**

- A statement is an instruction that the Python interpreter can execute.

- When you type a statement on the command line, Python executes it and displays the result, if there is one.

- The result of a print statement is a value. Assignment statements don't produce a result.

For example, the script

```
print 1
x = 2
print x
```

produces the output

```
1
2
```

statements and expressions

- **expressions**

- An expression is a combination of values, variables, operators and call to procedure. If you type an expression on the command line, the interpreter evaluates it and displays the result.

- Every expression ends in a new line.

a = b - a

b = b - a

a = b + a

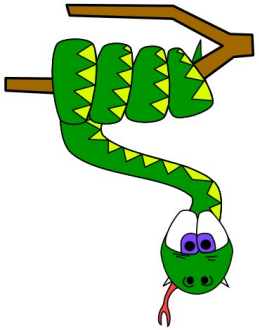
Multiple expressions are possible in one line by using semicolon character (;), but that is not by the PEP8 standard. The interpreter will not report an error

a = b - a; b = b -a; a = b + a

statements and expressions

- Long expressions can be divided into more lines by using \ character, except inside brackets - (), [], {}. In brackets, you don't need a special character like \ to divide expression into multiple lines because they clearly denote the start and end of a definition.

```
a = math.cos(b) + \
            math.cos(c)

arr=[10,20,
       30,
        40
     ]
```



indentation

CHINTECH

SMP

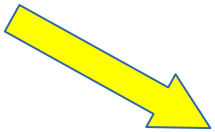
indentation

- The main diference between Python and other programming languages is that Python does not use any brackets to indicate blocks of code.

- Indentation is used to denote diferent blocks of code, such as the bodies of functions, conditionals, loops, and classes

- Instead of brackets, indentation is used

```
if condition:
    stetement1
    statement2                  # Consistent indentation
    statementN
else:
    stetement1
statement2                      # Inconsistent indentation (error)
    statementN
```



variables

Global Variables

- Variables that are created outside of a function (as in all of the examples above) are known as global variables.

- Global variables can be used by everyone, both inside of functions and outside.

```
x = "awesome"

def myfunc():

        print("Python is " + x)

myfunc()
```

variables

Global Variables

- If you create a variable with the same name inside a function, this variable will be local, and can only be used inside the function.

- The global variable with the same name will remain as it was, global and with the original value

```
x = "awesome"

def myfunc():

        x = "fantastic"

        print("Python is " + x)                    // Python is fantastic

myfunc()

print("Python is " + x)                             // Python is awesome
```

variables

Global Variables

➔ To create a global variable inside a function, you can use the **global** keyword.

```
def myfunc():
  global x
  x = "fantastic"
myfunc()
print("Python is " + x)
```

variables

nonlocal Variables

- The nonlocal keyword is used to work with variables inside nested functions, where the variable should not belong to the inner function

```
def outer():
    x = "local"
    def inner():
        nonlocal x
        x = "nonlocal"
        print("inner:", x)

    inner()
    print("outer:", x)

outer()
```

Output

```
inner: nonlocal
outer: nonlocal
```